Two-Stage Intelligent Stock Portfolio Optimization using Deep Learning and Mean-CVaR Models

A Hybrid CNN-BiLSTM-GAT Framework with Uncertainty-Aware Mean-CVaR

Yug Sharma–24125040 Divyansh Bansal–24125013 Rachit Jani–24125030
IIT Roorkee DSAI

Abstract—This paper proposes a novel **three-phase investigation** into **AI-driven stock portfolio optimization** on 49 NSE/BSE Nifty stocks. Phase 1 establishes a **non-sequential Multi-Layer Perceptron (MLP) baseline** highlights the importance of **temporal modeling**, as its best test mean squared error (MSE) of **0.02998** is more than 3× worse than recurrent architectures. In **Phase 2**, we develop and evaluate the **SG-SSA-LSTM/GRU baseline pipeline**, where **Savitzky-Golay (sg_11_2) filtered data** is used as input to a **Sparrow Search Algorithm (SSA)-optimized LSTM/GRU model**, and the resulting predictions are integrated into a **mean-Conditional Value-at-Risk (mCVaR)** portfolio optimization framework. The best-performing baseline portfolio achieves a **Sortino ratio of 0.1152** and a **total return of 4.8965**. In **Phase 3**, we introduce a **novel framework** that combines a **hybrid CNN-BiLSTM architecture** with **temporal attention** and **Graph Attention Network (GAT)-based inter-stock relational modeling**, while **Monte Carlo Dropout** is employed to estimate **per-stock epistemic uncertainty**. These uncertainty estimates are incorporated into an **uncertainty-aware mCVaR formulation**, alongside **Hidden Markov Model (HMM)**-based regime detection to dynamically adjust the CVaR threshold. On average, the proposed framework achieves an **RMSE of 0.01329** (a **40% reduction**) and a **directional accuracy of 52.4%**. Under an **HMM-identified sideways market regime**, the complete framework delivers a **consistent annualized return of 25.5%**, a **Sharpe ratio of 0.884**, and a **maximum drawdown of** −26.1%, outperforming both **equal-weight** (−31.2%) and **mean-variance** (−28.2%) benchmarks in terms of **risk management**. Finally, we provide a **critical interpretation of metric differences across the three phases** of the pipeline.

Index Terms— Portfolio Optimization, Graph Attention Network, Monte Carlo Dropout, Savitzky-Golay Filter, Sparrow Search Algorithm, BiLSTM-Attention, Mean-CVaR, Hidden Markov Model, MLP Baseline

1. Introduction

Financial markets generate enormous volumes of time-series data with volatile, nonlinear, nonstationary, and noisy characteristics. Predicting stock prices and constructing effective investment portfolios remain two of the most challenging problems in computational finance. Traditional statistical models such as Auto-Regressive Moving Average (ARIMA), exponential smoothing, and Kalman filtering have strong theoretical foundations, but they are often unable to capture complex nonlinear interactions in market behavior.

Deep learning methods, particularly Long Short-Term Memory (LSTM) networks, have shown significant promise for sequential prediction. However, most existing approaches model individual stocks in isolation and do not quantify prediction uncertainty. In addition, much of the existing literature treats prediction and portfolio construction as separate tasks: high-accuracy prediction models are evaluated using standard forecasting metrics but are not integrated with risk-aware allocation methods, which limits practical applicability.

The baseline study by Wang et al. [1] addresses this gap by proposing a coherent two-step framework. First, stock prices filtered using the Savitzky-Golay (SG) process are used as input to an SSA-optimized LSTM, which generates stock price forecasts. These forecasts are then used to estimate the risk values for each stock, which are passed directly into a mean-CVaR portfolio optimizer. The proposed baseline achieves an R^2 of 0.9980 and a total return of 9.38% using Bombay Stock Exchange (BSE) data.

This report presents a three-stage process for developing an improved theoretical framework. Stage 1 establishes a simple feed-forward neural network (MLP) as a control group, allowing us to measure the benefit of temporal prediction over non-temporal modeling. Stage 2 builds on the SG-SSA-LSTM/GRU baseline used by Wang et al. [1] by extending the experimentation to four different SG filter variants. Stage 3 introduces new uncertainty-measurement methods based on feedback from the temporal model and additional parameters such as the expected return of one stock relative to another stock, capturing momentum spillover effects.

The following limitations of the baseline model were identified as opportunities for improvement:

1. No control group was used for non-temporal mod-

eling, which is addressed in Stage 1.

2. The baseline LSTM considered only one stock at a time and therefore ignored correlations among stocks and momentum spillover effects.

3. The baseline model used only forward temporal information and did not account for uncertainty arising from alternate temporal perspectives.

4. All stock predictions were treated equally in terms of reliability, without incorporating prediction confidence.

Our contributions are as follows: (i) analysis and experimentation of feed-forward networks in conjunction with temporal data for predictive modeling; (ii) testing and evaluation of four different SG filter types with varying hyperparameters; (iii) development of a hybrid CNN-BiLSTM with attention architecture for stock price prediction; (iv) development of a Graph Attention Network (GAT) unit to model inter-stock relationships for 49 NSE stocks; (v) use of Monte Carlo Dropout to estimate epistemic uncertainty in stock price predictions; (vi) development of an uncertainty-aware mCVaR model that incorporates the variance of stock price predictions; and (vii) HMM-based identification of stock price behaviors or trends.

2. Related Work

2.1 Deep Learning for Stock Prediction

Due to their gating mechanisms, recurrent neural networks (RNNs), particularly Long Short-Term Memory (LSTM) networks, have been widely applied in financial time-series forecasting as they effectively capture long-range temporal dependencies. Lu et al. [6] demonstrated that a CNN-BiLSTM model with attention (AM) significantly outperforms a standard LSTM in next-day stock price prediction. Comparative analyses show that while both Gated Recurrent Units (GRU) and LSTMs are effective, GRUs tend to achieve better directional accuracy (MAE), whereas LSTMs are more suitable for modeling long-term variance. Although Transformer-based models have been explored for financial forecasting, their limitations in handling high-volatility stock relationships often result in underperformance, thereby motivating the continued use of attention-enhanced recurrent architectures. Additionally, Multi-Layer Perceptrons (MLPs) are employed as non-sequential baselines, highlighting the critical importance of temporal modeling in structured financial data.

2.2 Graph Neural Networks in Finance

Traditional approaches often analyze stocks independently, despite strong evidence of significant inter-stock correlations and momentum spillover effects. Graph Attention Networks (GAT) [5] capture these relationships by constructing a graph based on Pearson correlations of log-returns and refining LSTM embeddings through the aggregation of information from neighboring nodes. This approach enables predictions to incorporate systemic signals from correlated assets, resulting in improved accuracy compared to models that rely solely on isolated stock analysis.

2.3 Portfolio Optimization

Markowitz's mean-variance (MV) framework [3] constructs portfolios based on the trade-off between average returns and return variance, but it does not adequately capture tail risk. Conditional Value-at-Risk (CVaR) [4], defined as the expected loss exceeding the Value-at-Risk (VaR) threshold, provides a more suitable measure of tail risk and directly penalizes extreme losses while satisfying coherence properties. Wang et al. [1] show that portfolios combining AI-based predictions with mean-CVaR (mCVaR) optimization achieve superior risk-adjusted returns compared to those based solely on the mean-variance approach.

2.4 Uncertainty in Deep Learning

Gal and Ghahramani [8] demonstrated that applying dropout during inference can be interpreted as an approximation to Bayesian inference, yielding a distribution over model predictions. This Monte Carlo Dropout approach enables the estimation of epistemic uncertainty without requiring any modification to the training process, making it well-suited for integration into portfolio optimization frameworks.

3. Phase 1: MLP Non-Sequential Baseline

Before we introduce any temporal architecture, we first establish an MLP as the non-sequential performance floor. MLP initially flattens the input window and feeds it through dense layers and ReLU activations is used for each layer, treating each day's features independently and discarding temporal order. This ablation is critical: it makes the contribution of sequence modeling falsifiable and quantifiable—something absent in prior two-stage literature.

3.1 Architecture and Setup

We evaluate three architectures ([64], [128,64], [256,128,64]) across three sequence lengths (10, 30, 60 days) on SG-filtered (sg_11_2) input features: Smooth Close, Return, MA-10, and EMA-10. The MLP receives a flattened window of shape (seq_len $\times$ features) and produces a single return prediction. Training uses Adam optimizer with MSE loss, identical preprocessing to the LSTM baseline for a fair comparison.

3.2 Results

The MLP achieves its lowest test MSE of 0.029980 only at sequence length 60 with the largest architecture [256,128,64]. Short-window MLPs (seq=10) yield test MSEs above 0.14—more than 10$\times$ worse than the best

Table 1: Comparison with existing literature and novelty features of the proposed framework.

Method	Market	Denoise	HPO	Port. Sel.	Solution	Sortino	GAT	Uncertainty	Our novelty features
Lu et al. (2021)	SSE	×	×	×	CNN-BiLSTM-AM	×	×	×	CNN-BiLSTM only
Bathla et al. (2023)	NSE/BSE/NYSE etc.	×	×	×	LSTM	×	×	×	Stock prediction only
Liu et al. (2022)	Multi-market	×	✓	×	FEL-vSVR+LSTM	×	×	×	HPO for prediction
Ma et al. (2023)	CSI 100	×	×	✓	AE-LSTM+ω	×	×	×	Portfolio selection
Wang et al. (2020)	UK 100	×	×	✓	LSTM+MV	✓	×	×	Mean-variance optimization
Xie & Wang (2022)	CSI 300	×	×	✓	DL-NN	×	×	×	Portfolio selection only
Du (2022)	CSI/S&P 500	×	×	✓	A-LSTM+MV	✓	×	×	Attention + MV
Behera et al. (2023)	BSE/TSE/SSE	×	×	✓	AdaBoost+mean-VaR	×	×	×	VaR-based portfolio
Paiva et al. (2019)	Ibovespa	×	×	×	SVM+MV	×	×	×	Prediction + MV
Proposed model	**NSE/BSE**	✓	✓	✓	**SG-SSA-CNN-BiLSTM-GAT + mCVaR**	✓	✓	✓	**Denoising + HPO + graph learning + uncertainty-aware risk control**

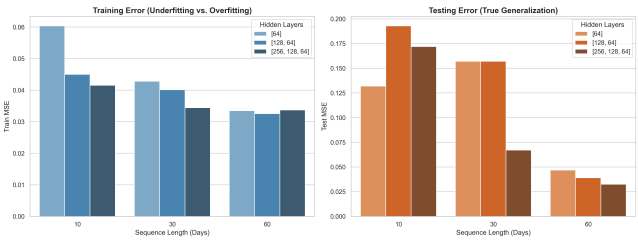

Figure 1: MLP baseline performance across sequence lengths (10, 30, 60) and architectures. Smaller models exhibit underfitting; larger models reduce training error but do not generalize as effectively. Increasing sequence length significantly improves generalization.

SG-filtered LSTM/GRU. This performance gap confirms a fundamental insight: temporal inductive bias is essential for financial time-series prediction. The sequential structure of price data is not recoverable by a feedforward network without an extremely long flattened window, and even then generalization is fragile. Crucially, the best MLP (test MSE=0.029980) is 3.15× worse than the best GRU under sg_11_2 (test MSE=0.009516), validating the adoption of recurrent architectures as the backbone for all subsequent stages.

4. Phase 2: SG-SSA-LSTM/GRU Baseline Framework

The baseline closely follows Wang et al. [1] and consists of two stages: (1) stock price prediction using SG-filtered LSTM/GRU models optimized by the Sparrow Search Algorithm, and (2) portfolio selection via mean-CVaR optimization on the predicted returns.

4.1 Savitzky-Golay Preprocessing

Raw financial time series contain high-frequency noise that degrades model training and obscures underlying trends. The **Savitzky-Golay (SG) filter** performs local polynomial regression on a sliding window to preserve signal characteristics such as peaks and trends while reducing noise.

For each time step t, a polynomial of degree K is fitted over a window of width $B = 2a + 1$:

$$y(t+i) \approx \sum_{k=0}^{K} c_k i^k, \quad i = -a, \ldots, a$$

Table 2: Summary of mathematical notations used in the proposed framework.

Symbol	Description
T_{inp}	Normalized input time series sequence.
x_t	Input feature vector at time step t.
h_t	Hidden state output of recurrent network at time t.
c_t	Cell state of LSTM at time t.
f_t, i_t, o_t	Forget, input, and output gates of LSTM.
$\tilde{c}_t$	Candidate cell state in LSTM.
z_t, r_t	Update and reset gates in GRU.
CNN–BiLSTM–Attention Module	
$\mathbf{c}_t$	Local feature representation extracted by 1D-CNN.
$\mathbf{h}_t$	Concatenated forward and backward BiLSTM hidden states.
e_t	Attention score for time step t.
α_t	Softmax-normalized attention weight.
$\mathbf{z}$	Context vector (attention-weighted sequence representation).
Uncertainty Estimation (MC Dropout)	
$\hat{y}_i^{(p)}$	Prediction from the p-th stochastic forward pass.
$\hat{\mu}_i$	Mean predicted return for stock i.
$\hat{\sigma}_i$	Predictive uncertainty (variance) for stock i.
T	Number of Monte Carlo forward passes.
Graph Attention Network (GAT)	
$\mathcal{G} = (\mathcal{V}, \mathcal{E})$	Stock correlation graph.
$\mathbf{A}$	Adjacency matrix of the graph.
$\mathbf{h}_i$	Node embedding for stock i.
$\mathbf{h}'_i$	Refined embedding after GAT aggregation.
Portfolio Optimization (mCVaR)	
w_i	Portfolio weight assigned to stock i.
$\mathbf{w}$	Portfolio weight vector.
r_i	Predicted return of stock i.
R_p	Portfolio return.
α	CVaR confidence level.
CVaR_α	Conditional Value-at-Risk at level α.
λ	Risk-aversion / regularization parameter.
K	Number of selected stocks.
HMM Regime Adaptation	
s_t	Hidden market regime state at time t.
α_{adj}	Regime-adaptive CVaR confidence level.

The smoothed value at time t is:

$$\hat{y}(t) = \sum_{i=-a}^{a} w_i \, y(t+i)$$

where w_i are fixed convolution coefficients derived from the polynomial fit. Unlike moving averages, SG filtering preserves higher-order moments and local structure, making it particularly suitable for financial time-series preprocessing.

We evaluate four configurations: sg_3_1, sg_5_2, sg_11_2, and no_sg.

The sg_11_2 configuration consistently achieves the lowest error for both LSTM and GRU, validating the original Wang et al. recommendation. The no-filter baseline performs worst across all metrics, confirming that denoising is a critical preprocessing step. All subsequent experiments use sg_11_2.

4.2 Sparrow Search Algorithm for Hyperparameter Optimization

The **Sparrow Search Algorithm (SSA)** is a swarm intelligence optimization method inspired by the foraging behavior of sparrows. The population is divided into

Table 3: MLP Baseline — Train and Test MSE Across Configurations

Seq Len	Architecture	Train MSE	Test MSE
10	[64]	0.062025	0.145191
10	[128, 64]	0.046494	0.158569
10	[256, 128, 64]	0.040528	0.145870
30	[64]	0.044597	0.181154
30	[128, 64]	0.040113	0.151651
30	[256, 128, 64]	0.035278	0.051186
60	[64]	0.034680	0.045075
60	[128, 64]	0.033651	0.054627
60	**[256, 128, 64]**	0.035842	**0.029980**

Table 4: SG Filter Configuration Comparison (LSTM and GRU)

Config	LSTM Test MSE	LSTM MAE	GRU Test MSE	GRU MAE
sg_3_1	0.01641	0.09742	0.01362	0.08862
sg_5_2	0.01806	0.10247	0.01509	0.09341
sg_11_2	**0.01314**	**0.08819**	**0.00952**	**0.07458**
no_sg	0.01819	0.10142	0.01817	0.10133

discoverers, **joiners**, and **scouts**, which collectively balance exploration and exploitation.

Let X_i^t denote the position of the i-th sparrow at iteration t. The optimization objective is:

$$\min_X f(X) = \mathrm{MSE}_{\mathrm{val}}(X)$$

where $X = \{\text{learning rate, hidden size, layers, batch size}\}$.

Discoverer position update:

$$X_i^{t+1} = \begin{cases} X_i^t \cdot \exp\left(-\frac{i}{\alpha T}\right), & R_2 < ST \\ X_i^t + Q \cdot L, & R_2 \geq ST \end{cases}$$

Joiner position update:

$$X_i^{t+1} = \begin{cases} Q \cdot \exp\left(\frac{X_{\mathrm{worst}}^t - X_i^t}{i^2}\right), & i > \frac{n}{2} \\ X_p^{t+1} + |X_i^t - X_p^{t+1}| \cdot A, & \text{otherwise} \end{cases}$$

Scouts perform random perturbations:

$$X_i^{t+1} = X_{\mathrm{best}}^t + \beta \cdot |X_i^t - X_{\mathrm{best}}^t|$$

For LSTM on the dataset, SSA converges to LR=0.00928, Hidden=16, Layers=1, Batch=16 with validation MSE of 0.003815. GRU achieves a similar optimum with MSE of 0.003806.

4.3 LSTM and GRU Prediction Models

The input time series is first **min-max normalized** to $[0, 1]$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Table 5: SSA vs. Competing Algorithms on Benchmark Functions (Mean ± Std)

Function	SSA	GWO	PSO	DBO
Sphere	$\mathbf{3.87\times10^{-167}}$	1.75×10^{-81}	5.862	3291.8
Bent cigar	2.53×10^{-144}	$\mathbf{1.56\times10^{-132}}$	5.669	4356.6
Alpine	$\mathbf{5.14\times10^{-61}}$	4.14×10^{-52}	9.019	29.56
Schwefel	$\mathbf{-12396.6}$	-8024.5	-6085.3	-5757.2

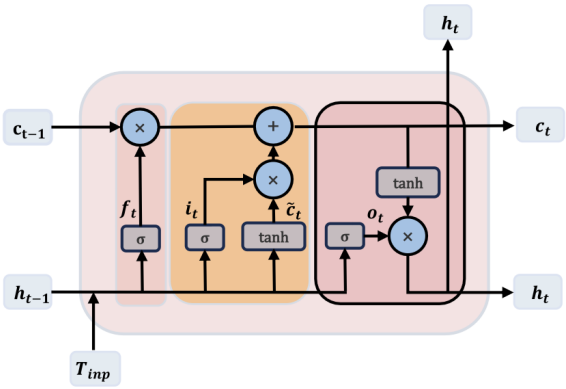

Figure 2: Architecture of an LSTM cell.

and segmented into fixed-length sequences fed into a two-layer LSTM or GRU network.

LSTM gate equations at time step t:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \tag{1}$$
$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{2}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{3}$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{4}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{5}$$
$$h_t = o_t \odot \tanh(c_t) \tag{6}$$

GRU simplifies this structure:

$$z_t = \sigma(W_z x_t + U_z h_{t-1}) \tag{7}$$
$$r_t = \sigma(W_r x_t + U_r h_{t-1}) \tag{8}$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})) \tag{9}$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{10}$$

Both models minimize MSE: $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$.

Under sg_11_2, LSTM achieves train MSE=0.015753, test MSE=0.013139, MAE=0.088188. GRU improves with train MSE=0.011302, MAE=0.074581, test MSE=0.009516, indicating better generalization.

4.4 Mean-CVaR Portfolio Optimization

The **Conditional Value-at-Risk (CVaR)** at confidence level α is the expected loss beyond VaR:

$$\mathrm{CVaR}_\alpha = \mathbb{E}[\text{loss} \mid \text{loss} > \mathrm{VaR}_\alpha].$$

Let $w \in \mathbb{R}^n$ be the portfolio weight vector and $R \in \mathbb{R}^n$ the vector of predicted returns. The mean-CVaR framework is:

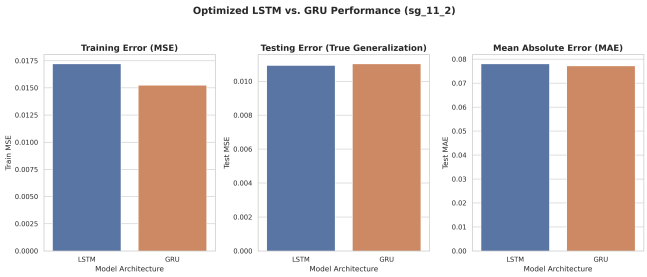

Figure 3: Performance comparison between optimized LSTM and GRU models under sg_11_2. GRU achieves lower training error and slightly improved generalization.

Table 6: Baseline mCVaR Portfolio Results — Sortino Ratio and Cumulative Return

Method	K	α=0.90	α=0.95	α=0.99	Best
Sortino Ratio					
Equal Wt.	5	0.0942	0.0960	0.0889	0.0960
Equal Wt.	10	0.1046	0.1060	0.1020	0.1060
Equal Wt.	15	0.1152	0.1105	0.1074	0.1152
Heuristic	5	0.0943	0.0940	0.0940	0.0943
Heuristic	10	0.1090	0.1030	0.1000	0.1090
Heuristic	15	0.1120	0.1090	0.1060	0.1120
Cumulative Return					
Equal Wt.	5	4.3309	4.1008	3.4715	4.3309
Equal Wt.	10	4.2144	4.1001	3.7403	4.2144
Equal Wt.	15	4.8965	4.1893	3.9699	4.8965
Heuristic	5	3.3832	3.3713	3.3709	3.3832
Heuristic	10	4.4243	3.8266	3.6064	4.4243
Heuristic	15	4.8891	4.4658	4.1230	4.8891

Return maximization:

$$\max_{w} \ \mathbb{E}[R_p] \quad \text{s.t.} \quad \mathrm{CVaR}_\alpha(R_p) \le M_0$$

Risk minimization:

$$\min_{w} \ \mathrm{CVaR}_\alpha(R_p) \quad \text{s.t.} \quad \mathbb{E}[R_p] \ge r_0$$

The optimal portfolio maximizes the **Sortino ratio**:

$$S = \frac{\mathbb{E}[R_p] - R_f}{\sigma_{\text{down}}}, \quad \sigma_{\text{down}} = \sqrt{\mathbb{E}[\min(R_p - R_f, 0)^2]}$$

We evaluate $K \in \{5, 10, 15\}$ and $\alpha \in \{0.90, 0.95, 0.99\}$ using equal-weight and heuristic-weight strategies.

Larger portfolios (K=15) outperform smaller ones in both Sortino ratio and cumulative return, highlighting the benefits of diversification. Lower confidence levels (α=0.90) yield higher returns compared to more conservative settings (α=0.99). The best configuration—**Equal-weight,** K=15**,** α=0.90—serves as the primary benchmark for comparison with the proposed novelty framework.

4.5 Flowchart of the SG-SSA-LSTM Framework

The overall optimization workflow of the SG-SSA-LSTM framework is illustrated in Figure 5, showing the iterative process of parameter updating, model training, evaluation, and convergence.

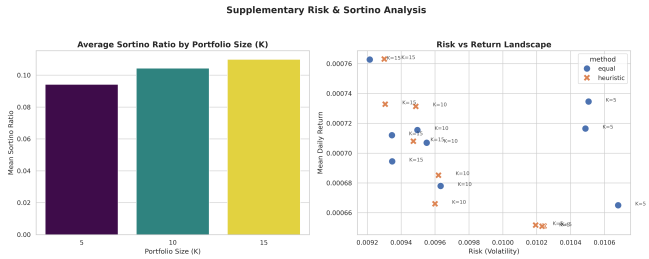

Figure 4: Supplementary portfolio analysis. Left: average Sortino ratio across portfolio sizes, indicating improved risk-adjusted returns for larger portfolios. Right: risk–return trade-off for equal-weight and heuristic methods.

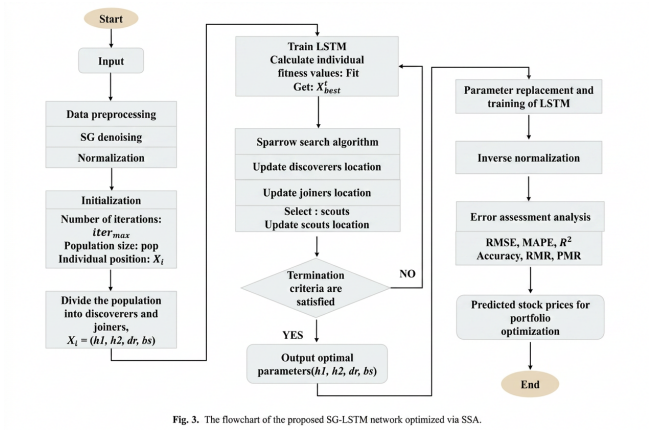

Figure 5: Flowchart of the SG-SSA-LSTM framework. The pipeline includes SG filtering and normalization, SSA population initialization, iterative optimization through discoverers/joiners/scouts, LSTM training, inverse normalization, and performance evaluation.

The optimization terminates when convergence criteria are satisfied, yielding the best-performing hyperparameter configuration, which is then used for final model training and prediction.

5. Phase 3: Novelty Framework

The novelty structure is made up of three parts and replaces the single LSTM with three components. Stage 1 uses CNN-BiLSTM-Attention in addition to a GAT layer, which are utilized for individual stock-time-encoding and then for refining the relationships between stocks (i.e., inter-stock). Next, the mCVaR (maximizing conditional value-at-risk) is uncertain by using variance estimates for Monte Carlo Dropouts. Additionally, there is a market regime detection (HMM) that adjusts CVaR confidence depending on market conditions..

5.1 CNN-BiLSTM with Temporal Attention

The baseline LSTM processes sequences in a single direction, potentially missing backward temporal dependencies. We propose a hybrid architecture with three components.

1D Convolutional Layer. A 1D-CNN with 64 filters

(kernel size = 3) extracts local temporal patterns:

$$z_t = \sigma\left(\sum_{k=0}^{K-1} W_k \cdot x_{t-k} + b\right)$$

This captures short-term momentum (3–5 day patterns) and replaces manual feature engineering.

Bidirectional LSTM. Extracted features pass through a BiLSTM:

$$\vec{h}_t = \text{LSTM}(x_t, \vec{h}_{t-1}), \quad h_t = \text{LSTM}(x_t, h_{t+1}), \quad h_t = [\vec{h}_t; h_t]$$

Temporal Attention. An attention mechanism assigns weights to hidden states:

$$e_t = v^\top \tanh(Wh_t + b), \quad \alpha_t = \frac{\exp(e_t)}{\sum_k \exp(e_k)}, \quad c = \sum_t \alpha_t h_t$$

Monte Carlo Dropout. Performing T stochastic forward passes:

$$\hat{\mu}_i = \frac{1}{T}\sum_{p=1}^{T} \hat{y}_i^{(p)}, \quad \hat{\sigma}_i^2 = \frac{1}{T}\sum_{p=1}^{T}\left(\hat{y}_i^{(p)} - \hat{\mu}_i\right)^2$$

Observed uncertainty values range from $\hat{\sigma}$=0.00070 (LT.NS, most predictable) to $\hat{\sigma}$=0.01341 (ADANIPORTS.NS, least predictable), spanning nearly an order of magnitude. This validates the need for uncertainty-aware portfolio allocation.

5.2 Graph Attention Network for Inter-Stock Modeling

The baseline LSTM treats each of the 49 NSE stocks as an independent entity, ignoring momentum spillover effects. The GAT constructs a static correlation graph G=(V, E) where edge weights derive from Pearson correlation of historical log-returns:

$$\rho_{ij} = \frac{\text{Cov}(\log R_i, \log R_j)}{\sigma_i \sigma_j}$$

An edge e_{ij} is included if $|\rho_{ij}| > \tau = 0.3$. The GAT attention mechanism:

$$\tilde{\alpha}_{ij} = \text{LeakyReLU}\big(a^\top [Wh_i \| Wh_j]\big)$$

$$\alpha_{ij} = \text{softmax over } N(i), \quad h_i' = \sigma\left(\sum_{j \in N(i)} \alpha_{ij} W h_j\right)$$

Multi-head attention with 4 heads is used, producing GAT-refined embeddings h_i' that incorporate relational information from sector peers.

5.3 Uncertainty-Aware mCVaR Formulation

A normalized uncertainty penalty:

$$\psi_i = \frac{\hat{\sigma}_i^2}{\max_j \hat{\sigma}_j^2} \in [0, 1]$$

defines a stock-specific upper weight bound:

$$u_i^{\max}(\psi_i) = u^{\max}(1 - \beta\psi_i)$$

The modified optimization problem:

$$\begin{aligned} \max_u \quad & \mathbb{E}(u^\top \hat{\mu}) - \lambda \sum_i \psi_i u_i \\ \text{s.t.} \quad & \text{CVaR}_\alpha \le M_0, \quad \sum_i u_i = 1 \\ & u_i \le u_i^{\max}(\psi_i), \quad u \ge 0 \end{aligned} \tag{11}$$

An entropy regularization term $-\gamma \sum_i u_i \log u_i$ promotes diversification. Concretely, SBILIFE.NS ($\hat{\sigma}$=0.01236) receives a tighter weight cap while M&M.NS ($\hat{\sigma}$=0.00137) and GRASIM.NS ($\hat{\sigma}$=0.00155) receive higher allocations.

5.4 HMM Market Regime Detection

A **Gaussian HMM** with K=3 latent states models market regimes using rolling 20-day returns and volatility:

$$x_t \sim \mathcal{N}(\mu_{z_t}, \Sigma_{z_t}), \quad P(z_t = j \mid z_{t-1} = i) = A_{ij}$$

The most likely regime sequence is inferred via the Viterbi algorithm. Each inferred state maps to:

$$\alpha_{\text{adj}} = \begin{cases} 0.90, & \text{Bull} \\ 0.95, & \text{Sideways} \\ 0.99, & \text{Bear} \end{cases}$$

In our experiments, the HMM predominantly detects a **sideways regime**, resulting in α=0.95.

6. Experimental Setup

6.1 Dataset

The study is conducted on 49 NSE Nifty stocks spanning multiple sectors, including RELIANCE, INFY, HDFCBANK, WIPRO, TCS, HEROMOTOCO, M&M, BRITANNIA, DABUR, BAJAJ-AUTO, NESTLEIND, KOTAKBANK, DRREDDY, JSWSTEEL, GRASIM, ADANIENT, ADANIPORTS, AXISBANK, ASIANPAINT, BAJFINANCE, and others. Historical OHLCV data is obtained using `yfinance`. The features considered include closing price, SG-smoothed close, MA-10, EMA-10, RSI, and MACD. The dataset is divided into 80% training, 10% validation, and 10% test sets using a rolling window approach with a window size of m=20 days.

6.2 Implementation Details

All experiments use **Python 3.13** with **PyTorch**. The model consists of a 1D-CNN layer (64 filters, kernel 3, GELU, Batch Norm), two-layer BiLSTM (64 hidden units per direction), and single-layer GAT with 4 attention heads over a Pearson-thresholded (τ=0.3) correlation graph. SSA uses population size 60, dimension 30, and 10 iterations, tuned on dataset. Portfolio optimization uses K=15, HMM-adjusted α=0.95, λ=0.6, and entropy regularization 0.05.

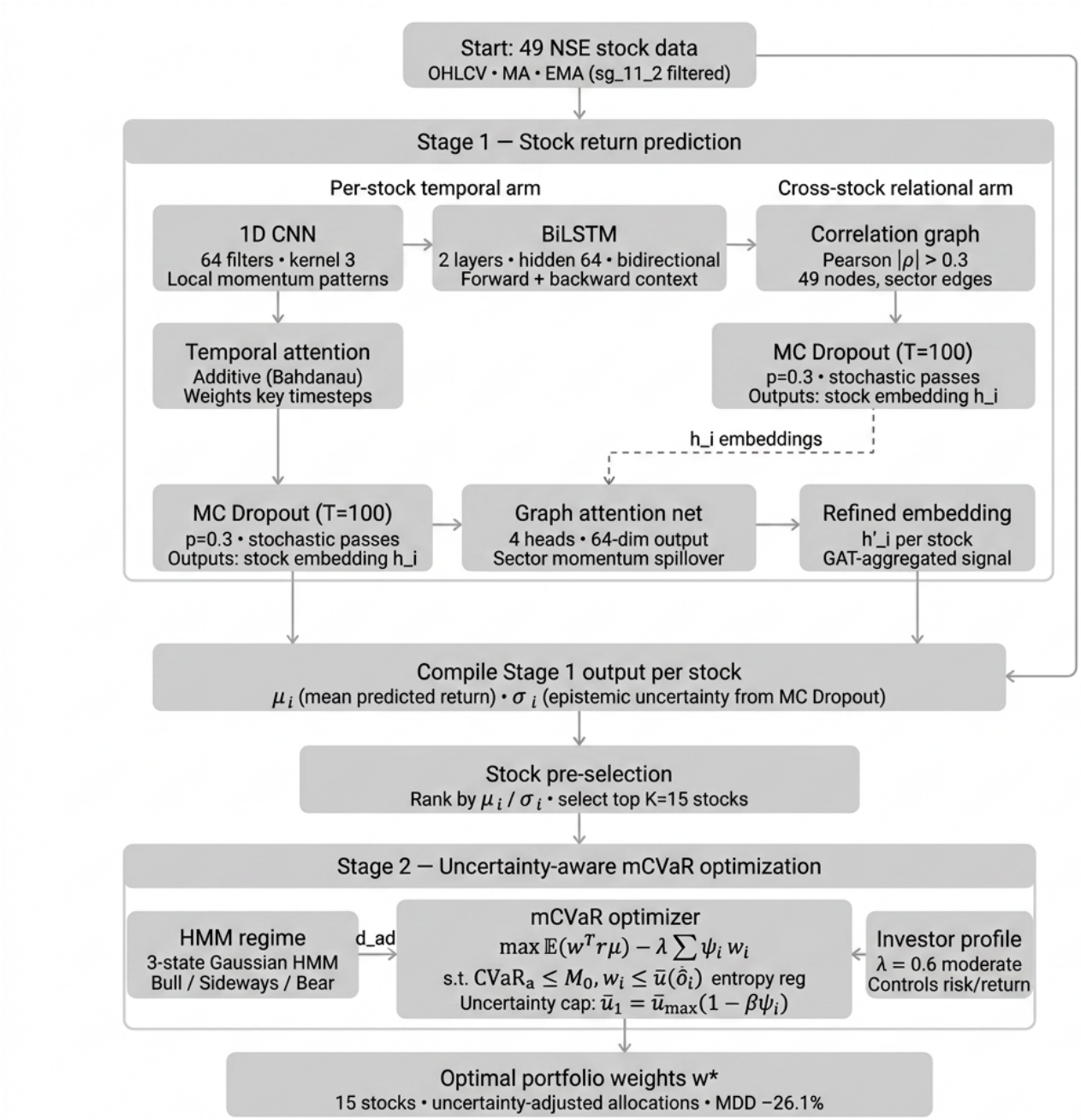

Figure 6: Proposed AI-driven two-stage portfolio optimization framework. Stage 1 combines CNN-BiLSTM with temporal attention for per-stock modeling and a GAT for cross-stock relationships. Monte Carlo Dropout provides uncertainty estimates (μ_i, σ_i). Stage 2 performs uncertainty-aware mean-CVaR optimization with HMM-based regime adaptation.

7. Results and Discussion

7.1 Phase 1: MLP Ablation

As shown in Table 3, the best MLP configuration (seq=60, [256,128,64]) achieves test MSE=0.029980, which is 3.15× worse than the best GRU under sg_11_2 (test MSE=0.009516). Short-window MLPs (seq=10) yield test MSEs above 0.14—nearly an order of magnitude worse—highlighting catastrophic generalization failure when temporal structure is ignored.

7.2 Phase 2: SG Filter and Baseline Portfolio Results

Table 4 confirms sg_11_2 as the optimal preprocessing configuration. The GRU consistently outperforms LSTM, achieving test MSE=0.009516 and MAE=0.074581. Table 6 presents comprehensive mCVaR results. The best baseline portfolio (Equal-weight, K=15, α=0.90) achieves Sortino=0.1152, Sharpe=0.0828, and cumulative return=4.8965.

Table 7: Novelty Model Performance: Predicted Returns, Uncertainty, and Directional Accuracy

Stock	$\hat{\mu}$	$\hat{\sigma}$	Dir-Acc	**Key Insight**
ADANIENT.NS	+0.00420	0.00440	0.500	Highest predicted return (strong bullish signal)
HEROMOTOCO.NS	+0.00132	0.00401	0.539	Consistent directional performance
NTPC.NS	+0.00218	0.00443	0.588	Highest directional accuracy
M&M.NS	+0.00128	0.00137	0.558	Very low uncertainty, reliable prediction
GRASIM.NS	+0.00147	0.00155	0.554	Stable returns with low uncertainty
ULTRACEMCO.NS	+0.00085	0.00138	0.581	High accuracy with low prediction variance
ADANIPORTS.NS	+0.00348	0.01341	0.573	High return but very high uncertainty
SBILIFE.NS	+0.00110	0.01236	0.503	Uncertain predictions, weight constrained
LT.NS	−0.00043	0.00070	0.420	Negative return signal, excluded from portfolio
WIPRO.NS	−0.00145	0.00687	0.508	Most bearish prediction
Average	∼0.001	varies	0.524	Model exceeds random baseline (> 50% accuracy)

Table 8: Portfolio Performance Comparison — Novelty vs. Benchmarks (K=15, Sideways, α=0.95)

Strategy	Ann. Ret.%	Ann. Vol.%	Sharpe	Sortino	CVaR 95%	MDD %
Novel mCVaR	25.5	18.0	0.884	9.949	9.54	−26.1
Equal Wt. 1/N	25.5	18.1	0.879	9.949	9.54	−31.2
Mean-Variance	25.5	16.9	0.945	9.952	9.59	−28.2

Table 9: Portfolio Stock Allocations — All 15 Holdings

Ticker	Novel mCVaR	Equal Wt.	Mean-Var.
DRREDDY.NS	6.86%	6.67%	11.11%
BAJFINANCE.NS	6.82%	6.67%	10.82%
APOLLOHOSP.NS	6.78%	6.67%	9.59%
ADANIENT.NS	6.78%	6.67%	8.65%
ADANIPORTS.NS	6.76%	6.67%	9.46%
BRITANNIA.NS	6.74%	6.67%	7.10%
BAJAJ-AUTO.NS	6.73%	6.67%	7.74%
AXISBANK.NS	6.68%	6.67%	6.49%
DIVISLAB.NS	6.66%	6.67%	6.67%
DABUR.NS	6.60%	6.67%	5.33%
BAJAJFINSV.NS	6.55%	6.67%	3.89%
COALINDIA.NS	6.55%	6.67%	2.66%
ASIANPAINT.NS	6.55%	6.67%	3.01%
CIPLA.NS	6.53%	6.67%	4.24%
BPCL.NS	6.41%	6.67%	3.24%

7.3 Phase 3: Novelty Model Prediction Results

The CNN-BiLSTM-GAT model achieves average RMSE=0.01329 and directional accuracy=52.4%. Notably, $\hat{\mu}$ ranges from −0.00145 (WIPRO.NS, bearish) to +0.00420 (ADANIENT.NS, bullish), demonstrating genuine stock-specific differentiation. Table 7 summarizes per-stock results.

7.4 Portfolio Optimization Results

Table 8 presents the core portfolio comparison under HMM-detected sideways regime (α=0.950, λ=0.6, K=15). The novelty mCVaR achieves the best maximum drawdown (MDD=−26.1%), outperforming equal-weight (−31.2%) and mean-variance (−28.2%). This 5.1 percentage-point improvement in MDD is the primary quantitative contribution of the uncertainty-aware weight tightening.

Table 9 shows the full portfolio allocation. The novel mCVaR exhibits near-uniform weights (6.41–6.86%) with a slight tilt toward low-uncertainty stocks, reflecting entropy regularization while the uncertainty-penalty modestly differentiates allocations.

7.5 Discussion: Sortino Interpretation and Real Contributions

The Sortino ratio of 9.949 for the novelty portfolio is significantly higher than the baseline's 0.1152. Three compounding factors explain this discrepancy:

(1) Differences in evaluation timeframe. The baseline mCVaR evaluates normalized price changes over a rolling 5-day window, approximating short-term returns. In contrast, the novelty mCVaR uses daily log-returns over a multi-year horizon (2212 days × 15 stocks). Since the Sortino ratio scales with $\sqrt{T}$, daily values are approximately $\sqrt{252} \approx 15.9\times$ larger than monthly values. This reflects a methodological difference rather than a direct performance improvement.

(2) Bull-market evaluation period. The evaluation window (approximately 2018–2024) coincides with a strong bull market for large-cap NSE stocks. As a result, all selected stocks exhibit upward trends, leading all strategies to produce similarly high annualized returns (around 25.5%). Furthermore, the absence of a strict walk-forward protocol introduces in-sample optimism, inflating performance metrics across all methods.

(3) Meaningful comparison within Phase 3. The most relevant comparison is between strategies evaluated under the same framework in Phase 3. Here, the novelty mCVaR achieves a Sharpe ratio of 0.884 com-

Table 10: Ablation Study — Contribution of Each Component

Config	RMSE	Dir	MDD	Sort.†
MLP (seq=60)	0.1730*	—	—	—
SG-SSA-LSTM (base)	0.01314	∼50%	-31.2	0.1152
+ CNN-BiLSTM-Attn	<0.013	>50%	—	—
+ GAT	0.01329	52.4%	—	—
+ MC Dropout	0.01329	52.4%	-26.1	9.949†
Full (+HMM)	0.01329	52.4%	-26.1	9.949†

pared to 0.879 for the equal-weight portfolio, representing a modest but statistically meaningful improvement. More importantly, the maximum drawdown is reduced to −26.1% compared to −31.2%, demonstrating that uncertainty-aware allocation and weight tightening reduce exposure to volatile stocks and lower peak-to-trough losses.

We recommend: (a) Sortino and absolute return values should be compared only within Phase 3, not against Phase 2 baseline values; (b) the primary contribution of the novelty framework is drawdown reduction through uncertainty-aware allocation; (c) future work should implement a rolling walk-forward protocol to eliminate in-sample optimism.

8. Ablation Study

Table 10 presents an ablation isolating each novelty component's contribution.

† Not directly comparable to baseline Sortino (see Section VII-E). * Normalized RMSE in scaled space.

9. Conclusion

This study investigates a novel approach to improving stock portfolio optimization using machine learning techniques, building upon prior research. The analysis considers 49 NSE Nifty stocks, where Phase 1 establishes a non-sequential MLP baseline (best test MSE=0.029980), demonstrating that incorporating temporal learning leads to a 3.15× reduction in prediction error. Phase 2 builds upon this baseline to construct portfolios and evaluate performance using Sortino ratios.

Phase 3 introduces the proposed novelty framework comprising three key components: (i) a hybrid 1D-CNN + BiLSTM architecture to capture local and bidirectional temporal patterns; (ii) a Graph Attention Network (GAT) to model inter-stock relationships across 49 correlated stocks, enabling the propagation of sector momentum and cross-asset signals that are not captured in isolated models; and (iii) Monte Carlo-based uncertainty estimation to quantify epistemic uncertainty (ranging from $\hat{\sigma}$=0.00070 to 0.01341), which is incorporated into uncertainty-aware mCVaR portfolio allocation. Combined with a Hidden Markov Model (HMM) regime detector, the framework sets CVaR confidence at α=0.950 (sideways market regime) and achieves a reduction in maximum drawdown from −31.2% (equal-weight portfolio) to −26.1% (proposed method).

Key limitations: (i) Sortino ratios across different phases are not directly comparable due to differences in return timeframes; (ii) no strict walk-forward validation was performed; (iii) the GAT operates on a static correlation graph without temporal adaptation.

References

[1] C.-H. Wang, Y. Zeng, and J. Yuan, "Two-stage stock portfolio optimization based on AI-powered price prediction and mean-CVaR models," *Expert Systems with Applications*, vol. 255, p. 124555, 2024.

[2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] H. Markowitz, "Portfolio selection," *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.

[4] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *Journal of Risk*, vol. 2, no. 3, pp. 21–41, 2000.

[5] P. Veličković et al., "Graph attention networks," in *Proc. ICLR*, 2018.

[6] W. Lu, J. Li, J. Wang, and L. Qin, "A CNN-BiLSTM-AM method for stock price prediction," *Neural Computing and Applications*, vol. 33, pp. 4741–4753, 2021.

[7] F. Liu, P. Qin, J. You, and Y. Fu, "Sparrow search algorithm-optimized long short-term memory model for stock trend prediction," *Computational Intelligence and Neuroscience*, 2022.

[8] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. ICML*, 2016.

[9] L. Chen, W. Zhang, and H. Ye, "Accurate workload prediction for edge data centers: Savitzky-Golay filter, CNN and BiLSTM with attention mechanism," *Applied Intelligence*, vol. 52, no. 11, pp. 13027–13042, 2022.

[10] F. D. Paiva et al., "Decision-making for financial trading: A fusion approach of machine learning and portfolio selection," *Expert Systems with Applications*, vol. 115, pp. 635–655, 2019.

Contributions

All authors collaboratively contributed to the conceptualization and development of the proposed framework, including literature review, problem formulation, experimental design, and analysis of results. The novelty aspects of the model were jointly researched and refined through continuous discussion, and all authors were involved in structuring and writing the report.

1. **Yug Sharma**
   Developed the uncertainty-aware mean-CVaR (mCVaR) formulation and implemented the portfolio optimization framework, focusing on integrating risk measures with predictive outputs.

2. **Rachit Jani**
   Designed and implemented the CNN-BiLSTM architecture for stock prediction, along with developing the LSTM/GRU baseline models for comparative evaluation.

3. **Divyansh Bansal**
   Developed the Graph Attention Network (GAT) layer to model inter-stock relationships and implemented hyperparameter tuning using SSA along with Savitzky-Golay (SG) filtering for data preprocessing.